Derive OFT and WBI Smart Contract Audit

Derive, 18 March 2026

hello@iosiro.com

Contents

1. Introduction 1
2. Disclaimer 3
3. Methodology 4
4. Audit findings 5
   - IO-DRV-OFT-001 Approved destinations are chain-agnostic 7
   - IO-DRV-OFT-002 bytes32(0) can be approved as destination 8
   - IO-DRV-OFT-003 Single DVN compromise could lead to infinite mint 9
   - IO-DRV-OFT-004 Rate limit underflow DoS on limit reduction 10
   - IO-DRV-OFT-005 Fixed-window rate limiter allows double extraction at boundary 11
   - IO-DRV-OFT-006 Missing _disableInitializers() in constructor 12
   - IO-DRV-OFT-007 renounceOwnership() is callable 13
   - IO-DRV-OFT-008 Single-step ownership transfer 14
   - IO-DRV-OFT-009 No upper bound on approvedDestinations array 15
   - IO-DRV-OFT-010 Rate limit disabled when bucketWidth = 0 16
   - IO-DRV-OFT-011 approvalRequired() returns true despite mint/burn pattern 17
5. Code quality improvement suggestions 18
6. Specification 19
   - DeriveOFTReceiver 19
   - WithdrawBridgeIntent 19
7. Test coverage report 20

1. Introduction

iosiro was commissioned by Derive to perform a smart contract audit of the `Derive OFTReceiver` and `WithdrawBridgeIntent` smart contracts. Two auditors conducted the audit between 10 and 13 March 2026, using 3 audit days.

Overview

The audit focused on two components of Derive's cross-chain bridging infrastructure: the `DeriveOFTReceiver` contract, a LayerZero OFT implementation using a mint/burn pattern for cross-chain token transfers, and the `WithdrawBridgeIntent` contract, an intent-based withdrawal bridge for Derive L2.

No critical or high-risk issues were identified. Five low-risk issues were reported, primarily related to rate-limiting edge cases, single-DVN reliance, and destination approval design. Six informational issues covered defensive coding practices such as disabling initializers, using two-step ownership transfers, and rate limit validation.

All issues were acknowledged by Derive and closed, as these contracts are intended for short-term use, and the risk was deemed acceptable. The table below shows the reported findings' status at the assessment's conclusion.

	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Resolved	0	0	0	0	0
Closed	0	0	0	5	6

Scope

The assessment focused on the source files listed below, with all other files considered out of scope. Any out-of-scope code interacting with the assessed code was presumed to operate correctly without introducing functional or security vulnerabilities.

- **Project name:** derive-bridges
- **Audit commit:** `3cd8d29`
- **File:** `contracts/DeriveOFTReceiver.sol`
- **Project name:** v2-aa
- **Audit commit:** `7a41278`
- **File:** `src/intents/WithdrawBridgeIntent.sol`

A specification is available in the Specification section of this report.

2. Disclaimer

This report aims to provide an overview of the assessed smart contracts' risk exposure and a guide to improving their security posture by addressing identified issues. The audit, limited to specific source code at the time of review, sought to:

- Identify potential security flaws.
- Verify that the smart contracts' functionality aligns with their documentation.

Off-chain components, such as backend web application code, keeper functionality, and deployment scripts, were explicitly not in scope of this audit.

Given the unregulated nature and ease of cryptocurrency transfers, operations involving these assets face a high risk from cyber attacks. Maintaining the highest security level is crucial, necessitating a proactive and adaptive approach that accounts for the experimental and rapidly evolving nature of blockchain technology. To encourage secure code development, developers should:

- Integrate security throughout the development lifecycle.
- Employ defensive programming to mitigate the risks posed by unexpected events.
- Adhere to current best practices wherever possible.

3. Methodology

The audit was conducted using the techniques described below.

Code review	The source code was manually inspected to identify potential security flaws. Code review is a useful approach for detecting security flaws, discrepancies between the specification and implementation, design improvements, and high-risk areas of the system.
Dynamic analysis	The contracts were compiled, deployed, and tested in a test environment, both manually and through the test suite provided. Dynamic analysis was used to identify additional edge cases, confirm that the code was functional, and validate the reported issues.
Automated analysis	Automated tooling was used to detect the presence of various types of security vulnerabilities. Static analysis results were reviewed manually, and any false positives were removed. Any true positive results are included in this report.

4. Audit findings

The table below provides an overview of the audit's findings. Detailed write-ups are provided below.

ID	Issue	Risk	Status
IO-DRV-OFT-001	Approved destinations are chain-agnostic	Low	Closed
IO-DRV-OFT-002	bytes32(0) can be approved as destination	Low	Closed
IO-DRV-OFT-003	Single DVN compromise could lead to infinite mint	Low	Closed
IO-DRV-OFT-004	Rate limit underflow DoS on limit reduction	Low	Closed
IO-DRV-OFT-005	Fixed-window rate limiter allows double extraction at boundary	Low	Closed
IO-DRV-OFT-006	Missing _disableInitializers() in constructor	Informational	Closed
IO-DRV-OFT-007	renounceOwnership() is callable	Informational	Closed
IO-DRV-OFT-008	Single-step ownership transfer	Informational	Closed
IO-DRV-OFT-009	No upper bound on approvedDestinations array	Informational	Closed
IO-DRV-OFT-010	Rate limit disabled when bucketWidth = 0	Informational	Closed
IO-DRV-OFT-011	approvalRequired() returns true despite mint/burn pattern	Informational	Closed

Each issue identified during the audit has been assigned a risk rating. The rating is determined based on the criteria outlined below.

Critical risk The issue could result in the theft of funds from the contract or its users.

High risk The issue could result in the loss of funds for the contract owner or its users.

Medium risk The issue resulted in the code being dysfunctional or the specification being implemented incorrectly.

Low risk A design or best practice issue that could affect the ordinary functioning of the contract.

Informational An improvement related to best practice or a suboptimal design pattern.

In addition to a risk rating, each issue is assigned a status:

Open The issue remained present in the code as of the final commit reviewed and may still pose a risk.

Resolved The issue was identified during the audit and has since been satisfactorily addressed, removing the risk it posed.

Closed	The issue was identified during the audit and acknowledged by the developers as an acceptable risk without actioning any change.

IO-DRV-OFT-001

Approved destinations are chain-agnostic

Low	Closed	WithdrawBridgeIntent#L67-75

The `approvedDestinations` mapping stores only a raw `bytes32` recipient. As such, approvals are not tied to a specific chain. That means the system can't express "approved for Ethereum but not Solana," and it relies on the executor and off-chain flow to pair a destination with the correct `destEID`. While not an approval bypass, the current implementation does create some user-intent ambiguity in a multi-chain setup, and using the correct `bytes32` destination with the wrong chain `destEID` could result in the permanent loss of the funds being withdrawn.

Recommendation

Consider scoping approved destinations by `(destination, destEID)` as an additional guardrail.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-002

bytes32(0) can be approved as destination

Low	Closed	WithdrawBridgeIntent.sol#L67-L75

There was no validation to prevent the approval of a zero address as a destination. Bridged tokens would be burnt on Derive L2 and minted to the zero address on the destination chain, effectively burning them permanently. The risk is limited since this is a self-inflicted attack; the SCW owner must explicitly call `addApprovedDestination(bytes32(0))`.

Recommendation

Add `require(destination != bytes32(0))` in `addApprovedDestination` to prevent users accidentally bridging without a valid recipient address.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-003

Single DVN compromise could lead to infinite mint

Low	Closed	DeriveOFTReceiver.sol#L146-L158

The DVN configuration relies on a single DVN (LayerZero Labs) with no additional DVNs. If that DVN is compromised, an attacker could forge messages to mint an unlimited number of OFT tokens on Derive L2 and try to withdraw them. This attack would bypass the existing rate limits and enable the attacker to drain the Derive bridge contract deposits.

Recommendation

Both `_credit()` and `_debit()` should be pausable, and an off-chain DVN honesty monitor should be deployed. The off-chain honesty monitor could pause the functions if it detects a compromised DVN trying to mint and then withdraw assets that aren't backed by messages from source chains. Additional DVNs that support Derive L2 should also be used, requiring an N-of-N DVNs to agree before processing a message. Care should be taken when selecting additional DVNs to prevent the bridge freezing due to DVN outages.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-004

Rate limit underflow DoS on limit reduction

Low	Closed	DeriveOFTReceiver.sol#L100

If the owner uses `setWithdrawLimit()` to reduce `maxWithdrawLimit` below `currentWithdrawnAmount`, the subtraction in `_checkRateLimit()` underflows, causing a Solidity Panic (`0x11`). The panic blocks all non-whitelisted withdrawals until the current window resets. The `getAvailableWithdrawLimit()` view handles this case correctly; only `_checkRateLimit()` is affected.

Recommendation

In `setWithdrawLimit()`, if `_maxWithdrawLimit` is less than the `currentWithdrawnAmount`, the limit should be set to `currentWithdrawnAmount` as a floor. Alternatively, a `>=` guard could also be added to `_checkRateLimit()` before the subtraction to return zero available limit instead of reverting.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-005

Fixed-window rate limiter allows double extraction at boundary

Low	Closed	DeriveOFTReceiver.sol#L93-L96

The rate limiter for `DeriveOFTReceiver` and `WithdrawBridgeIntent` uses a fixed limit per window. When a window expires, its limit is reset. An attacker could attempt to withdraw the full limit just before window `n` expires, wait one block for the reset, then withdraw the full limit of window `n+1`, effectively doubling the configured rate. The total extraction remains bounded by `lockedAmount`.

Recommendation

A shifting window would remove the ability to extract double the limit around a window boundary.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-006

Missing _disableInitializers() in constructor

Informational	Closed	DeriveOFTReceiver#L31-34

The `DeriveOFTReceiver` constructor does not call `_disableInitializers()`, allowing the implementation contract to be initialized directly. This has no security impact at the moment, but it is a deviation from OpenZeppelin's recommended pattern.

Recommendation

Add `_disableInitializers()` to the `DeriveOFTReceiver` constructor.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-007

renounceOwnership() is callable

Informational	Closed	IntentExecutorBase.sol#L13

`Ownable.renounceOwnership()` is inherited and not overridden in `IntentExecutorBase`. Calling it would permanently brick all admin functions with no recovery path.

Recommendation

The `renounceOwnership()` function should be overridden to revert.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-008

Single-step ownership transfer

Informational	Closed	IntentExecutorBase.sol#L13, WithdrawBridgeIntent.sol#L43

The `IntentExecutorBase` inherits OZ `Ownable` rather than `Ownable2Step`. A wrong address in `transferOwnership()` would cause an irrecoverable loss of all admin functions.

Recommendation

The `IntentExecutorBase` contract should inherit `Ownable2Step` instead of `Ownable`.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-009

No upper bound on approvedDestinations array

Informational	Closed	WithdrawBridgeIntent.sol#L67-L75

An SCW could approve an unbounded number of destinations. When the array of destinations is too large, gas costs for checking the array may cause Out of Gas (OOG) errors. The risk is limited since the SCW would be only griefing itself and could remove earlier array elements to recover from the grief.

Recommendation

Add a reasonable cap (e.g., `MAX_APPROVED_DESTINATIONS = 20`) to limit the number of destinations an SCW may approve.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-010

Rate limit disabled when bucketWidth = 0

Informational	Closed	WithdrawBridgeIntent.sol#L229-L238

`setBucketParams(0, N)` with N >= 1 effectively disables rate limiting because the bucket resets on every call. This is purely an owner misconfiguration, and the default state (0, 0) is safe.

Recommendation

Add the `require(_bucketWidth > 0 || _maxWithdrawPerBucket == 0)` validation to `setBucketParams()`. This prevents a bucket width of zero while also having a max larger than zero.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

IO-DRV-OFT-011

approvalRequired() returns true despite mint/burn pattern

Informational	Closed	DeriveOFTReceiver.sol#L14

The inherited `approvalRequired()` returns `true`, corresponding to a transfer-based lock/unlock pattern. However, `DeriveOFTReceiver` uses a mint/burn pattern via `burn(from, amount)`. This mismatch may cause front-end integrations to prompt users for unnecessary ERC-20 approvals.

Recommendation

`approvalRequired()` should be overridden to return `false`.

Client response

The issue was acknowledged. Derive indicated that the contracts are intended for short-term use, and the risk was deemed acceptable.

5. Code quality improvement suggestions

Code improvement suggestions without security implications are listed below.

#	Location	Details
1	WithdrawBridgeIntent.sol#L168, WithdrawBridgeIntent.sol#L206	Both execute functions in `WithdrawBridgeIntent` use `safeIncreaseAllowance()` to approve the wrapper, but the tokens are always transferred in and fully consumed. `forceApprove()` could be used instead, as it is equivalent and saves the extra `allowance()` SLOAD that `safeIncreaseAllowance()` performs.
2	WithdrawBridgeIntent.sol#L167-L178, WithdrawBridgeIntent.sol#L205-L219	Reorder checks before `safeTransferFrom()` in both execute functions. Currently, `safeTransferFrom()` executes before the fee check and recipient validation. If either check reverts, the gas spent on the transfer is wasted. Moving `FeeTooHigh` / `InvalidRecipient` checks before the transfer saves gas on revert paths.
3	WithdrawBridgeIntent.sol#L70, WithdrawBridgeIntent.sol#L113	In `addApprovedDestination()` and `isApprovedDestination()`, `destinations.length` is read from storage on every loop iteration. Caching it in a local variable avoids repeated warm SLOADs. `removeApprovedDestination()` already caches the length.
4	WithdrawBridgeIntent.sol#L70, WithdrawBridgeIntent.sol#L84, WithdrawBridgeIntent.sol#L113	The loop index `i` in all three functions that check the approved destinations cannot overflow. Using `unchecked { ++i }` skips the overflow check and saves gas.
5	WithdrawBridgeIntent.sol#L81-L94	When removing the last destination using `removeApprovedDestination()`, the loop copies the last element onto itself before popping. Add the `if (i != length - 1)` condition before copying to prevent this case.
6	WithdrawBridgeIntent.sol#L105-L110	The `checkSCWOwner` parameter in `isApprovedDestination()` is undocumented in the Natspec comment for the function. It controls whether the SCW owner is implicitly accepted as a valid destination (EVM chains). The missing `@param checkSCWOwner` tag should be added to the comment.
7	WithdrawBridgeIntent.sol#L254-L255	The errors `DestinationAlreadyApproved` and `InvalidB32Address` are declared but never used, suggesting incomplete refactoring. The `DestinationAlreadyApproved` error should be thrown before the early return in `addApprovedDestination()` and the `InvalidB32Address` error can be deleted.

All of the above improvements were acknowledged. Derive indicated that the contracts are intended for short-term use and the improvements were deemed unnecessary.

6. Specification

The following section outlines the audited contracts' intended functionality at a high level, based on their implementation in the codebase. Any perceived points of conflict should be highlighted with the auditing team to determine the source of the discrepancy.

DeriveOFTReceiver

`DeriveOFTReceiver` inherits the LayerZero OFT (Omnichain Fungible Token) implementation and enables cross-chain token bridging using a mint/burn pattern. When tokens are sent from a source chain, they are burned, and a corresponding amount is minted on the destination chain via a LayerZero message verified by DVNs (Decentralized Verifier Networks). The contract includes a rate limiter that restricts the volume of withdrawals within configurable time windows up to the total credited amount per chain. It also supports whitelisted addresses that bypass rate limits but are still beholden to the max limit of the total credited amount. The contract is deployed behind an upgradeable proxy and uses OpenZeppelin's initializer pattern.

WithdrawBridgeIntent

`WithdrawBridgeIntent` is an intent-based withdrawal bridge that allows Smart Contract Wallet (SCW) owners on Derive L2 to bridge assets to approved destinations on other chains. SCW owners maintain a list of approved destination addresses. Withdrawals are executed by an executor that pulls the tokens from the sender, sends them via the OFT bridge, and collects a fee. The contract inherits from `IntentExecutorBase`, which provides ownership management. Rate limiting is implemented via a bucketing system with configurable width and maximum withdrawal per bucket.

7. Test coverage report

The test coverage report for the `WithdrawBridgeIntent` contract is provided below:

File	% Lines	% Statements	% Branches	% Funcs
src/intents/WithdrawBridgeIntent.sol	79.41% (54/68)	77.33% (58/75)	69.23% (9/13)	83.33% (10/12)

No tests were available for `DeriveOFTReceiver` at the time of the audit. Adding tests for `DeriveOFTReceiver` and improving the coverage of `WithdrawBridgeIntent` with high-quality tests would contribute toward improving the overall security posture of the contracts.